

# Tìm số chiều tối thiểu bảo toàn khoảng cách

Sử dụng Scikit-Learn để tính toán theo JL lemma:

```
1 from sklearn.random_projection import johnson_lindenstrauss_min_dim
2
3 m = 5000 # so truong hop
4 eps = 0.1 # dung sai
5
6 d = johnson_lindenstrauss_min_dim(m, eps=eps)
7 print(d)
8 # Output: 7300
9
```

# Phép chiếu ngẫu nhiên thủ công

Tạo một ma trận  $P$  lấy mẫu từ phân phối chuẩn và nhân trực tiếp:

```
1 import numpy as np
2
3 n = 20_000 # so chieu ban dau
4 m = 5_000  # so mau
5 d = 7300   # chieu muc tieu
6
7 # Tao ma tran ngau nhien P
8 P = np.random.randn(d, n) / np.sqrt(d)
9
10 X = np.random.randn(m, n)
11
12 # Thuc hien giam chieu
13 X_reduced = X @ P.T
14
```

# Scikit-Learn Random Projections

Dễ dàng hơn với 2 lớp xây dựng sẵn:

```
1 from sklearn.random_projection import GaussianRandomProjection
2 from sklearn.random_projection import SparseRandomProjection
3
4 # 1. Gaussian Random Projection
5 gaussian_rnd_proj = GaussianRandomProjection(eps=0.1, random_state=42)
6 X_reduced = gaussian_rnd_proj.fit_transform(X)
7
8 # 2. Sparse Random Projection
9 sparse_rnd_proj = SparseRandomProjection(eps=0.1)
10 # Hoạt động tương tự nhưng ma trận chiều hình thành chủ yếu bằng số 0.
11 # Giúp tiết kiệm RAM cực mạnh (vd 25MB thay vì 1.2GB)
```

# Nhúng tuyến tính cục bộ (Locally Linear Embedding - LLE)

- Là một kỹ thuật **Giảm chiều phi tuyến tính (NLDR)** và thuộc họ **Học đa tạp (Manifold Learning)**.
- Không sử dụng phép chiếu như PCA.
- **Cơ chế:**
  - 1 Đo lường cách mỗi điểm liên kết tuyến tính với các điểm láng giềng gần nhất của nó.
  - 2 Tìm biểu diễn chiều thấp sao cho mỗi liên kết (khoảng cách cục bộ) này được **bảo toàn tốt nhất**.
- Tuyệt vời để mở cuộn các đa tạp bị xoắn (như Swiss roll) nếu dữ liệu không có quá nhiều nhiễu.

# Sử dụng LocallyLinearEmbedding cho Swiss roll

```
1 from sklearn.datasets import make_swiss_roll
2 from sklearn.manifold import LocallyLinearEmbedding
3
4 # Tao du lieu Swiss roll 3D
5 X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)
6
7 # Giam xuong 2D bang LLE, danh gia tren 10 lang gieng gan nhat
8 lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
9 X_unrolled = lle.fit_transform(X_swiss)
10
```

(Giả lập Hình 8-10: Dữ liệu Swiss Roll 3D được "mở cuộn" thành một mặt phẳng 2D thành công, duy trì quan hệ giữa các điểm bên cạnh nhau).

# Thuật toán LLE: Bước 1 (Mô hình hóa cục bộ)

- Đối với mỗi điểm  $x^{(i)}$ , LLE tìm  $k$  láng giềng gần nhất của nó.
- Sau đó cố gắng **tái tạo điểm**  $x^{(i)}$  như một hàm tuyến tính (tổng có trọng số) của  $k$  láng giềng đó.
- Mục tiêu: Tìm các trọng số  $w_{i,j}$  sao cho khoảng cách bình phương giữa  $x^{(i)}$  và  $\sum w_{i,j}x^{(j)}$  là nhỏ nhất có thể.
- Kết thúc bước 1, ta có một ma trận trọng số  $W$  phản ánh chính xác cấu trúc hình học **cục bộ** của tập dữ liệu trong không gian gốc (chiều cao).

## Thuật toán LLE: Bước 2 (Giảm chiều bảo toàn khoảng cách)

- Ánh xạ các điểm vào không gian  $d$  chiều (với  $d < n$ ).
- Giữ nguyên ma trận trọng số  $W$  đã tìm ở Bước 1.
- Tìm các vị trí điểm mới (tọa độ mới) trong không gian  $d$  chiều, sao cho chúng **vẫn thỏa mãn ma trận trọng số** đó càng gần càng tốt.
- *Điểm yếu của LLE*: Mở rộng kém cho tập dữ liệu rất lớn.

# Các kỹ thuật giảm chiều khác: MDS & Isomap

- **MDS (Multidimensional Scaling):** Cố gắng giảm chiều trong khi bảo toàn **khoảng cách vật lý** giữa các trường hợp như ở không gian gốc. Làm phẳng thay vì mở cuộn.
- **Isomap:** Xây dựng đồ thị nối các láng giềng, sau đó giảm chiều trong khi bảo toàn **khoảng cách trắc địa (geodesic distance)** giữa các điểm. (Trắc địa là đi qua các điểm kết nối, không phải bay đường chim bay). Thích hợp mở cuộn đa tạp.

# Các kỹ thuật giảm chiều khác: t-SNE & LDA

- **t-SNE (t-Distributed Stochastic Neighbor Embedding):** Giảm chiều bằng cách giữ các điểm tương đồng ở gần nhau, nhưng đẩy các điểm dị biệt ra thật xa. Đặc biệt tốt để **phát hiện cụm trực quan** (như vẽ bản đồ MNIST 2D).
- **LDA (Linear Discriminant Analysis):** Một thuật toán **Phân loại** (Classification). Trong quá trình huấn luyện nó sẽ học trực phân biệt rõ rệt nhất giữa các lớp. Bạn dùng trực này để chiếu dữ liệu, giúp giữ cho các nhãn cách xa nhau nhất có thể → Tiềm xử lý tuyệt vời cho thuật toán phân loại khác.

# Tổng kết Chương 8

- **Khái niệm & Động lực:** Giảm số chiều dữ liệu giúp phòng tránh "Lời nguyền số chiều", tăng tốc mô hình và khả năng trực quan hóa.
- **PCA:** Phương pháp Phép chiếu, sử dụng SVD, là kỹ thuật mặc định ưu tiên số một. Nắm vững Incremental PCA (cho dữ liệu lớn) và Randomized PCA.
- **Random Projection:** Vô địch về tốc độ và tiết kiệm RAM cho dữ liệu siêu lớn mà không cần huấn luyện.
- **Manifold Learning (LLE, Isomap...):** Có khả năng "mở cuộn" các hình khối xoắn ốc phức tạp, thích hợp cho dữ liệu có cấu trúc đa tạp phi tuyến tính.

**CẢM ƠN CÁC BẠN ĐÃ LẮNG NGHE!**

Q & A